

Empacotamento Bidimensional com Conflitos

Um *solver* guloso e BRKGA para minimizar recipientes

Ycaro Sales

18 de junho de 2026

- 1 Introdução
- 2 Definição do problema
- 3 Representação do problema
- 4 Heurísticas utilizadas
- 5 Resultados

A importância da logística no mundo atual

- A logística é a espinha dorsal da economia moderna: transporte, armazenamento e distribuição respondem por boa parte dos custos operacionais de empresas.
- O **e-commerce** multiplicou o volume de remessas e a pressão por entregas rápidas e baratas.
- Acomodar bem as mercadorias — em caixas, paletes, contêineres e caminhões — é decisivo para a competitividade.

Por que minimizar recipientes?

Cada contêiner, palete ou caixa economizada significa **menos viagens, menos área de estoque, menos combustível e menor impacto ambiental**. Otimizar o empacotamento gera ganho econômico e sustentabilidade.

Empacotamento Bidimensional (2D Bin Packing) com Conflitos

Entrada:

- Recipientes (faixas) retangulares de largura W e altura H fixas;
- n itens retangulares, cada um com largura w_i , altura h_i e classe de cliente c_i ;
- Um **grafo de conflitos** $G = (\mathcal{R}, E)$.

Objetivo: empacotar *todos* os itens minimizando o número de recipientes utilizados.

Restrições:

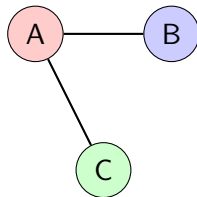
- Sem sobreposição; item inteiramente dentro do recipiente;
- Itens ligados por uma aresta em G **não podem** dividir o mesmo recipiente;
- Permite-se rotação de 90° .

O que são “conflitos”?

Dois itens *em conflito* não podem viajar no mesmo recipiente. Exemplos reais:

- Produtos químicos incompatíveis;
- Mercadorias de clientes ou rotas diferentes;
- Itens que, por segurança/regulação, não podem ser transportados juntos.

O problema é **NP-difícil**: generaliza o *Bin Packing* e se relaciona à coloração de grafos (cada cor $\approx$ um recipiente).



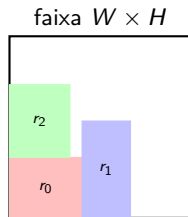
A–B e A–C em conflito

Como o problema é modelado

Uma **instância** reúne todos os dados de entrada:

- **Faixa:** dimensões fixas $W \times H$;
- **Itens:** lista de retângulos, cada um com largura w_i , altura h_i e classe de cliente c_i ;
- **Grafo de conflitos** G : indexado pela *posição* de cada item, com um nó por item e uma aresta por par incompatível.

As instâncias são lidas de arquivos JSON; a demanda de cada tipo de item é expandida em itens individuais.



Representação da solução

Uma solução é o conjunto de **faixas**, cada uma com a posição (origem, dimensões e orientação) dos itens nela alocados.

Representação geométrica — Container + EMS

- Cada faixa mantém seus **Espaços Máximos Vazios** (retângulos livres maximais) e as colocações já feitas;
- Inserção pela regra **bottom-left** (menor origem em y , depois em x), recortando os EMS interceptados;
- Checagem de **conflito**: rejeita o item se um vizinho seu em G já está na faixa.

Representação para a busca — cromossomo BRKGA

Vetor de $2n$ **chaves aleatórias** em $[0, 1]$: as primeiras n codificam a *ordem de inserção* e as últimas n a *orientação* de cada item (detalhado adiante).

Heurística gulosa: First-Fit-Decreasing (linha de base)

- ① Ordena os itens por **área decrescente**;
- ② Insere cada item por **First-Fit** nas faixas abertas;
- ③ Abre uma nova faixa quando nenhuma aberta aceita o item.

Detalhes geométricos

- Inserção **ciente de conflitos** (rejeita item se um vizinho em G já está na faixa) e com **auto-rotação**.
- Posicionamento via **Espaços Máximos Vazios** (EMS): o espaço livre é um conjunto de retângulos vazios maximais, recortados a cada inserção (regra *bottom-left*).

Biased Random-Key Genetic Algorithm: cada solução é um vetor de $2n$ chaves aleatórias em $[0, 1]$.

- Primeiras n chaves $\rightarrow$ **ordem de inserção** (ordenação crescente);
- Últimas n chaves $\rightarrow$ **orientação** ($\geq 0,5 \Rightarrow$ rotacionado).

Um **decodificador** determinístico empacota por First-Fit honrando os conflitos. Aptidão a *minimizar*:

$$f = \text{faixas_usadas} + \text{nao_alocados} \times (n + 1)$$

A penalidade $(n + 1)$ garante que toda solução inviável seja pior que qualquer viável.

Parâmetros: população 100; parada com 200 gerações estagnadas; seleção por torneio; *crossover* uniforme; mutação de gene único.

BRKGA: decodificação chaves $\rightarrow$ solução

Exemplo com $n = 3$ e chaves $[0.9, 0.1, 0.5 \mid 0.2, 0.5, 0.8]$:

Ordem (1ª metade): $\text{item1} < \text{item2} < \text{item0} \Rightarrow [1, 2, 0]$



Orientação (2ª metade): $\text{item0}=\text{normal}, \text{item1}=\text{rotacionado}, \text{item2}=\text{rotacionado}$



First-Fit ciente de conflitos $\Rightarrow$ faixas com itens posicionados

Apenas o decodificador conhece o problema; seleção, *crossover* e mutação operam genericamente sobre as chaves.

Resultados: gulosa vs. BRKGA (18 instâncias)

Instância	n	Gulosa	BRKGA
CJCM-E02N20_Hard_c0.3	20	6	4
CJCM-E00N23_HardInfeasible_c0.5	23	8	6
CJCM-E03X18_HardFeasible_c0.1	18	3	2
Total (18 instâncias)		91	78

- BRKGA: **melhor em 11**, empate em 7, **nunca pior**.
- Redução média de $\approx$ **14,3%** no número de faixas.
- Tempos: gulosa $< 0,01$ s; BRKGA $\approx 0,07$ – $0,13$ s por instância.

- Os sufixos $c0.1$, $c0.3$, $c0.5$ indicam a probabilidade de conflito entre pares de itens.
- Mais conflitos $\Rightarrow$ menos itens podem dividir um recipiente $\Rightarrow$ **mais faixas necessárias**.
- É justamente nos cenários mais restritos que o BRKGA mais se distancia da heurística gulosa — evidenciando o valor da busca metaheurística.

Conclusão

Combinando representação por chaves aleatórias (BRKGA), decodificador First-Fit ciente de conflitos e gestão de espaços livres por EMS, obtivemos soluções consistentemente melhores que a heurística gulosa, sem nunca piorar nenhuma instância.

Trabalhos futuros

- Incorporar a restrição de **descarregamento** por classe de cliente;
- Avaliar sobre um conjunto maior de instâncias;
- Ajuste sistemático dos parâmetros da metaheurística.

Obrigado!

Perguntas?